Alvine GPU Implementation Audit

Complete source-inspection report · IPPL repository · PDF edition

Repository: /home/raven/Documents/vic/my_implementation/ippl

alvine contains substantial GPU-portable computation: particle initialization and movement, grid stencils, spectral operations, time integration, diagnostics, and spectrum accumulation all use Kokkos kernels. It also makes explicit GPU-conscious choices for NUFFT algorithms, FFT communication, device data retention, and CUDA debugging.

The underlying allocation, kernel dispatch, interpolation, MPI packing, and FFT/NUFFT engines are inherited from IPPL/Kokkos. GPU execution still depends on the configured backend and compatible dependencies/MPI. This is a source inspection, not a GPU compilation or runtime validation. No repository files were modified during the inspection.

Snippets are excerpts; … denotes omitted code. “Implemented in alvine” identifies where the implementation resides, without asserting Git authorship. Source paths are relative to the repository root; line numbers refer to the inspected source snapshot.

Contents

1. Decisions implemented in alvine: storage and execution policies
2. Particle initialization, grid initialization, remeshing, and movement
3. Grid and spectral computation
4. FFT/NUFFT choices made in alvine
5. Time integration and retention of device state
6. Reductions, atomics, MPI boundaries, and synchronization
7. Explicit host-transfer and CUDA-debug boundaries
8. GPU-portable tests and incomplete/legacy implementations
9. Functionality inherited from IPPL/Kokkos

1. Decisions implemented in alvine: storage and execution policies

1. Field and particle aliases

alvine/datatypes.h:24 — Field, Field_t, VField_t, ParticleAttrib

```
using Field = ippl::Field<T, Dim, Mesh_t<Dim>, Centering_t<Dim>, ViewArgs...>;

using ParticleAttrib = ippl::ParticleAttrib<T>;
```

Uses IPPL's View-backed containers rather than host arrays. ViewArgs... allows field storage properties to propagate. Actual execution and memory spaces come from IPPL/Kokkos; these aliases do not explicitly force HostSpace.

2. Compatible spectral storage and transform types

alvine/core/AlvineManager2D.hpp:43; alvine/core/AlvineManager3D.hpp:49 — spectral type aliases

```
using Nufft_t = ippl::FFT<ippl::NUFFTransform, RealField_t>;

using ComplexField_t = typename Nufft_t::ComplexField;

using SpectralFft_t = ippl::FFT<ippl::CCTransform, ComplexField_t>;
```

Keeps spectral fields compatible with the FFT wrapper's storage and backend selection. alvine does not convert spectral state into host std::complex arrays for computation.

3. 2D component arrays and registered integration state

alvine/particles/ParticleContainer2D.hpp:16 — attributes and registerAttributes()

```
ippl::ParticleAttrib<T> omega;

ippl::ParticleAttrib<T> ux;

ippl::ParticleAttrib<T> uy;

this->addAttribute(rk4_R0);

this->addAttribute(rk4_k1);
```

Separate scalar arrays for strength and velocity components provide a structure-of-arrays representation for component transforms. Positions and stage vectors use vector-valued attributes. Registered RK4 state participates in inherited particle resizing and migration.

4. 3D vector state plus scalar transform components

alvine/particles/ParticleContainer3D.hpp:17 — attributes and registerAttributes()

```
particle_position_type omega;

ippl::ParticleAttrib<T> omega_x;

ippl::ParticleAttrib<T> omega_y;

ippl::ParticleAttrib<T> omega_z;

this->addAttribute(rk4_omega0);

this->addAttribute(rk4_omega_k1);
```

Maintains both vector-valued particle state and separate scalar component arrays consumed by NUFFT. Gradients, viscosity, base states, and slopes remain managed attributes rather than host scratch vectors. This costs extra storage and packing, but avoids host staging and keeps stage data aligned during migration.

5. View-specific policies

alvine/spectral/AlvineGridVelocity.hpp:16 — computeVelocityField(); also diagnostics and Euler/Strang kernels

```
Kokkos::parallel_for("Assign rhs",

    ippl::getRangePolicy(view, nghost), KOKKOS_LAMBDA(…));
```

The policy follows the View’s execution space and excludes ghost cells. This is stronger than relying solely on Kokkos’s global default execution space.

6. Multidimensional default-backend policies

alvine/FslManager.h:306 — initializeGridVorticity(); alvine/spectral/AlvineSpectralOps3D.hpp:328 — computeSpectralVelocityModes3D()

```
Kokkos::MDRangePolicy<Kokkos::Rank<2>>(…)

Kokkos::MDRangePolicy<Kokkos::Rank<3>>(…)
```

Grid and mode loops are multidimensional parallel launches. These policies omit an explicit execution space, so they use Kokkos’s default backend. That supports GPU builds but assumes storage is accessible from that backend. No custom tile sizes or iteration orders are selected here.

7. Count-based particle kernels and device lambdas

alvine/VortexInCellManager.h:178 — initializeParticles(); repeated throughout managers

```
Kokkos::parallel_for("init_particle_positions", nlocal,

    KOKKOS_LAMBDA(const int i) { … });
```

Particle loops use the count-based parallel interface, which dispatches through Kokkos's default execution space. KOKKOS_LAMBDA makes the body callable on the selected device and captures local values/View handles for the launch.

8. Extracting Views and scalars before launching

alvine/FslManager.h:339 — pushVirtualParticlesForward(); alvine/spectral/SFSLCoupledStrang3D.hpp:44 — setCoupledStrangStage3D()

```
auto R_view = pc->R.getView();

auto P_view = pc->P.getView();

double dt = this->dt_m;

… KOKKOS_LAMBDA(…) {

    R_view(p) = R_view(p) + P_view(p) * dt;

}
```

Extracts Views and scalar parameters before launching. Device code accesses device-compatible handles instead of dereferencing shared_ptr containers or the host manager object. Most current kernels follow this pattern.

9. CUDA extended-lambda access accommodation

alvine/spectral/SFSLStrang3D.hpp:4; alvine/spectral/SFSLCoupledStrang3D.hpp:4 — included kernel launchers

```
// Included in the public section … CUDA extended

// lambdas require public kernel launchers.
```

An explicit accommodation for CUDA extended-lambda compilation. Kernel-launching member functions are placed in the public section rather than treated as ordinary private helpers.

2. Particle initialization, grid initialization, remeshing, and movement

10. Device-side 2D VIC lattice generation

alvine/VortexInCellManager.h:178 — initializeParticles()

```
unsigned ix_local = i % nxp_local;

unsigned iy_local = i / nxp_local;

R_view(i)[0] = x;

R_view(i)[1] = y;
```

Computes lattice positions directly in particle Views. Rank-local bounds are prepared on the host, but the per-particle work runs in parallel without uploading a host-built position array. Its second kernel initializes strengths in the same way.

11. Device-side 2D Fourier particle generation

alvine/VortexInFourierManager.h:194 — initializeParticles()

```
Kokkos::parallel_for("init_particle_positions", nlocal,

    KOKKOS_LAMBDA(…));

Kokkos::parallel_for("init_particle_vorticity", nlocal,

    KOKKOS_LAMBDA(…));
```

The Fourier particle initializer also generates positions and strengths directly in the execution space.

12. Device-side 3D Taylor–Green particles

alvine/VortexInFourier3DManager.h:2123 — initializeParticles(); alvine/SpectralFSL3DManager.h:260 — resetVirtualParticlesToGridFromTGV3D()

```
"init_taylor_green_3d_particles"

"initialize_sfsl3d_tgv_virtual_particles"

KOKKOS_LAMBDA(const size_t p) { … }
```

Generates 3D lattice state and evaluates the analytic Taylor–Green condition inside particle kernels. No serial host initialization of every particle is required.

13. Stateless, device-callable perturbations

alvine/SinusoidalJitter.hpp:8 — sinusoidalPositionJitter(), sinusoidalVorticityPerturbation()

```
KOKKOS_INLINE_FUNCTION

double seed = Kokkos::sin(…) * 43758.5453;

double random01 = seed - Kokkos::floor(seed);
```

Helpers are callable from device kernels and use Kokkos mathematical functions. Perturbations are computed from global indices without a host RNG, mutable random pool, or random-number transfer. Backend transcendental differences can still change numerical values.

14. Device-callable analytic reference functions

alvine/test_cases/TaylorGreen2D.hpp:15; alvine/test_cases/TaylorGreen3D.hpp:15 — analytic functions and fillVorticity()

```
KOKKOS_INLINE_FUNCTION

return Kokkos::sin(x) * Kokkos::cos(y) * Kokkos::cos(z);

Kokkos::parallel_for("fill_taylor_green_3d_vorticity", …);
```

Analytic reference fields can be evaluated on the device. The field-fill routines use 2D/3D MDRange kernels instead of host loops followed by copies.

15. Parallel exact fields in the reference manager

alvine/PerfectInviscid2DManager.h:66 — exact_vorticity(), exact_velocity()

```
"perfect_inviscid_2d_exact_vorticity"

"perfect_inviscid_2d_exact_velocity"

KOKKOS_LAMBDA(const int li, const int lj) { … }
```

Even this analytic/reference manager fills grid Views in parallel, preserving a device execution path.

16. Direct grid-to-particle initialization with local indexing

alvine/FslManager.h:232; alvine/SpectralFSLManager.h:250 — initializeVirtualParticles()

```
int li = ii + nghost;

int lj = jj + nghost;

omega_p(p) = omega_g(li, lj) * dA;
```

```
P_view(p) = u_g(li, lj);
```

Reads field Views and writes particle Views within one kernel. Local View indices are separated from global physical indices, which matters for decomposed GPU-resident fields. Assigning both strength and velocity in the same pass also avoids an extra gather for this initialization path.

17. Rank-local grid initialization

alvine/FslManager.h:286; alvine/SpectralFSLManager.h:305 — initializeGridVorticity()

```
MDRangePolicy<Rank<2>>({nghost, nghost},

    {nghost + …, nghost + …})

int j = j0 + lj - nghost;
```

Launches over locally allocated interior cells and converts local indices to global coordinates inside the kernel. This avoids host field construction and supports MPI decomposition.

18. 2D remeshing in particle kernels

alvine/VortexInCellManager.h:310; alvine/VortexInFourierManager.h:309 — remeshParticlesFromGrid()

```
"remesh_vic_particles_from_grid"

"remesh_vif_particles_from_spectral_grid"

int grid_i = static_cast<int>(

    Kokkos::floor((x - rmin[0]) / hr[0]));
```

Position reconstruction, index calculation, and grid sampling happen inside particle kernels. Remeshing does not require copying the whole grid to the host.

19. 3D remeshing and spectral sample packing

alvine/VortexInFourier3DManager.h:742 — remeshParticlesFromGrid3D(), sampleRemeshedParticlesFromSpectralModes3D(); alvine/SpectralFSL3DManager.h:345 — createGridLatticeParticles3D(), sampleGridParticlesFromSpectralModes3D()

```
"place_vif_3d_remesh_lattice_particles"

"pack_remeshed_vif_3d_type2_samples"

"place_sfsl3d_grid_lattice_particles"

"pack_sfsl3d_type2_vorticity_samples"
```

3D remeshing uses device placement and packing kernels, with inherited type-2 NUFFT evaluation for spectral samples. Separate recovery kernels prepare physical Fourier coefficients without host mode loops.

20. Parallel particle pushes

alvine/FslManager.h:354; alvine/SpectralFSLManager.h:384 — pushVirtualParticlesForward(); alvine/SpectralFSL3DManager.h:544 — pushVirtualParticlesForward3D()

```
R_view(p) = R_view(p) + P_view(p) * dt;
```

One parallel iteration updates each particle’s position. Distinct output particles avoid write races and require no atomics.

21. Device periodic wrapping

alvine/VortexInFourier3DManager.h:2155; alvine/SpectralFSL3DManager.h:615 — wrapParticlePositions3D()

```
Kokkos::parallel_for("wrap_vif_3d_particle_positions", n,

    KOKKOS_LAMBDA(…) { … });
```

Periodic wrapping is performed on the particle Views in a device kernel rather than a host traversal. Subsequent ownership migration remains an IPPL operation.

22. Device deletion masks

alvine/VortexInCellManager.h:223 — clearParticles(); alvine/FslManager.h:368 — clearVirtualParticles(); spectral counterparts

```
Kokkos::View<bool*> invalid("invalid", nlocal);

KOKKOS_LAMBDA(…) { invalid(p) = true; }

Kokkos::fence();

pc->destroy(invalid, nlocal);
```

Builds the deletion mask in default-space device-accessible storage. It passes that mask directly to inherited particle destruction, avoiding a host flag array. The fence completes marking before destruction.

3. Grid and spectral computation

23. Parallel finite-difference velocity stencil

alvine/spectral/AlvineGridVelocity.hpp:4 — computeVelocityField()

```
omegaField.fillHalo();

view(i,j) = {

    (omega_view(i,j+1)-omega_view(i,j-1))/(2*hr(1)),

    -(omega_view(i+1,j)-omega_view(i-1,j))/(2*hr(0))

};
```

Computes the finite-difference velocity stencil directly in field Views. Halo exchange supplies neighboring values; the bulk stencil remains parallel. The halo mechanism is inherited.

24. 2D Fourier Poisson/Biot–Savart algebra

alvine/spectral/AlvineSpectralOps2D.hpp:41 — computeSpectralVelocityModes2D()

```
const auto psi = omega(i,j) / (area * k2);

ux(i,j) = imag * derivativeKy * psi;

uy(i,j) = -imag * derivativeKx * psi;
```

The Fourier Poisson/Biot–Savart algebra runs per mode in an MDRange kernel using Kokkos::complex. It avoids a host spectral solver loop. Zero/Nyquist handling is inside the same kernel.

25. 3D strength splitting and spectral preconditioning

alvine/spectral/AlvineSpectralOps3D.hpp:4 — spectralScatter3D()

```
"split_omega_components_3d"

nufftType1_mp->transform(pc.R, pc.omega_x, omega_x_hat_m);

"precondition_spectral_vorticity_modes_3d"
```

Splits vector strengths into scalar attributes on the device, invokes three inherited type-1 transforms, and applies spectral preconditioning in a 3D kernel. Component conversion is not host staging.

26. Parallel solenoidal projection

alvine/spectral/AlvineSpectralOps3D.hpp:216 — projectSpectralVorticityModes3D()

```
const auto correction = kDotOmega / k2;

ox(i,j,k) -= kx * correction;

oy(i,j,k) -= ky * correction;

oz(i,j,k) -= kz * correction;
```

Performs the solenoidal projection independently per local mode. Each iteration owns its output cell, so no global atomic accumulation or host projection is needed.

27. Fused 3D velocity component kernel

alvine/spectral/AlvineSpectralOps3D.hpp:291 — computeSpectralVelocityModes3D()

```
ux(i,j,k) = imag * (ky * oz(i,j,k) - kz * oy(i,j,k));

uy(i,j,k) = imag * (kz * ox(i,j,k) - kx * oz(i,j,k));
```

Computes all velocity components together in one per-mode kernel. The stored vorticity modes have already been preconditioned; this is the code’s storage convention, not an omitted denominator.

28. Spectral gather with device component packing

alvine/spectral/AlvineSpectralOps2D.hpp:160 — spectralGather2D(); alvine/spectral/AlvineSpectralOps3D.hpp:474 — spectralGather3D()

```
nufftType2_mp->transform(pc.R, pc.ux, ux_hat_m);

KOKKOS_LAMBDA(…) {

    P(i)[0] = ux(i); P(i)[1] = uy(i); …

}
```

Evaluates scalar spectral components through inherited NUFFT and packs them into vector particle velocity on the device. Optional shape filtering also runs in an MDRange kernel.

29. Device Hou–Li filtering

alvine/spectral/AlvineSpectralOps2D.hpp:280; alvine/spectral/AlvineSpectralOps3D.hpp:121 — Hou_Li_filter()

```
Kokkos::exp(-T(alpha) * Kokkos::pow(eta, exponent))

view(…) *= filterFactor;
```

Evaluates and applies the filter on the device using device-portable math. There is no host filter table to transfer each step.

30. Parallel two-thirds mode truncation

alvine/spectral/AlvineSpectralOps3D.hpp:168 — twoThirdsFilter3D()

```
if (Kokkos::abs(T(mx)) > cutoffX || …) {

    view(i,j,k) = Kokkos::complex<T>(0.0,0.0);

}
```

Implements mode truncation as a parallel local operation.

31. Device shape-function initialization and application

alvine/spectral/AlvineSpectralOps2D.hpp:323 — initializeShapeFunctionVIF();
alvine/spectral/AlvineSpectralOps3D.hpp:432 — initializeShapeFunctionVIF3D(); shape-application functions

```
Sk *= Kokkos::pow(arg, order);

Skview(…) = Sk;

omegaView(…) *= shapeView(…);
```

Builds and applies B-spline Fourier factors in execution-space kernels. The 2D implementation explicitly notes adaptation from Alpine's InitializeShapeFunctionPIF; its implementation resides in alvine.

32. Device spectral velocity damping

alvine/spectral/AlvineSpectralOps2D.hpp:106 — applySpectralVelocityViscosity2D();
alvine/spectral/AlvineSpectralOps3D.hpp:375 — 3D counterpart

```
const T viscousFactor = Kokkos::exp(-nu * k2 * dt);

ux(…) *= viscousFactor;
```

Applies viscous damping per spectral mode on the device, updating multiple components together.

33. 2D spectral viscosity and particle update

alvine/spectral/AlvineSpectralOps2D.hpp:367 — computeSpectralViscosity(), spectralGatherViscosity2D(), updateParticleVorticityViscosity()

```
visc(i,j) = -T(isNotZero) * viscosity_m * k2 * omega(i,j) / area;

nufftType2_mp->transform(pc.R, pc.viscosity, visc_hat);
```

```
omega(i) += dt * visc(i) * particleArea;
```

Spectral viscosity, sampling, and strength updates all use the View/NUFFT path. The direct viscosity_m member access in the first kernel is a portability caveat, discussed in section 8.

34. 3D spectral viscosity and particle update

alvine/spectral/AlvineSpectralOps3D.hpp:552 — computeSpectralViscosityModes3D(), spectralGatherViscosity3D(), applyParticleViscosity3D()

```
viscX(i,j,k) = -nu * k2 * k2 * ox(i,j,k);

visc(p)[0] = viscX(p);

omega(p)[0] += dt * visc(p)[0] * particleVolume;
```

Uses spectral kernels, three component NUFFTs, device packing, and per-particle updates. The extra k2 matches the preconditioned 3D mode representation.

35. Device velocity gradients and vortex stretching

alvine/spectral/AlvineSpectralOps3D.hpp:880 — computeSpectralVelocityGradientModes3D(), spectralGatherGradientModes3D(), applyParticleVortexStretching3D()

```
"compute_spectral_velocity_gradient_modes_3d"

nufftType2_mp->transform(pc.R, pc.duxdx, duxdxModes);

"apply_particle_vortex_stretching_3d"
```

Computes nine gradient mode fields in parallel, evaluates them into scalar particle attributes, and forms stretching in a particle kernel. Host code orchestrates the transforms; numerical loops remain on the execution backend.

36. Device cell-center phase corrections

alvine/spectral/AlvineSpectralOps2D.hpp:240 — applyCellCenteredIfftPhase();
alvine/spectral/AlvineSpectralOps3D.hpp:682 — phase helpers

```
const Kokkos::complex<T> factor(

    Kokkos::cos(phase), Kokkos::sin(phase));

view(…) *= factor;
```

Applies the cell-center phase correction directly to spectral Views before/after FFT. The 3D implementation shares a signed-phase kernel between forward and inverse corrections.

37. Reconstruction without host spectral arrays

alvine/spectral/AlvineSpectralOps2D.hpp:437; alvine/spectral/AlvineSpectralOps3D.hpp:732 — reconstruction functions

```
auto omegaModes = omega_hat_m.deepCopy();

spectralFft_mp->transform(ippl::BACKWARD, omegaModes);

omegaOut(i,j) = omegaGrid(i,j).real();
```

Reconstruction uses same-space temporary fields, inherited FFT, and device conversion from complex modes to real/vector grid values. Velocity reconstruction also removes the mean with a kernel rather than a host traversal.

4. FFT/NUFFT choices made in alvine

38. Explicit native NUFFT selection

alvine/spectral/AlvineSpectralSetup2D.hpp:8 — initNUFFT2D(); alvine/spectral/AlvineSpectralSetup3D.hpp:42 — rebuildNUFFTPlans3D()

```
p1.add("use_finufft", false);

p2.add("use_finufft", false);
```

Explicitly selects IPPL’s native Kokkos NUFFT implementation. This avoids dependence on a host FINUFFT execution path. The native implementation itself is inherited.

39. Tiled spreading and sorted gathering

Same spectral setup functions

```
p1.add("spread_method", "tiled");

p2.add("gather_method", "atomic_sort");
```

Selects tiled spreading and sorted gathering. Tiled spreading uses inherited team scratch storage to reduce global atomic traffic; sorted gathering improves spatial access locality. Despite its name, AtomicGather does not itself issue atomics.

40. Pruned native transform path

Same spectral setup functions

```
p1.add("use_upsampled_inputs", false);

p2.add("use_upsampled_inputs", false);
```

Uses the native pruned-transform path instead of requiring externally supplied full upsampled fields. This also matches the native type-2 path documented for distributed execution.

41. Delegating spectral FFT plan defaults

alvine/spectral/AlvineSpectralSetup2D.hpp; alvine/spectral/AlvineSpectralSetup3D.hpp — initNUFFT2D(), initNUFFT3D()

```
fftParams.add("use_heffte_defaults", true);

spectralFft_mp = std::make_shared<SpectralFft_t>(FL, fftParams);
```

Delegates FFT backend and plan options to IPPL/heFFTe. In the inspected IPPL adapter, this option also enables GPU-aware communication and pipelined point-to-point reshaping.

42. Explicit GPU-aware Poisson FFT settings

alvine/FieldSolver.hpp:63 — initFFTSolver()

```
sp.add("use_heffte_defaults", false);

sp.add("use_pencils", true);

sp.add("use_reorder", false);

sp.add("use_gpu_aware", true);

sp.add("comm", ippl::p2p_pl);

sp.add("r2c_direction", 0);
```

Explicit GPU-aware FFT communication request, pencil decomposition, pipelined point-to-point communication, and fixed transform direction. These are plan choices; they do not establish that the runtime MPI supports device pointers.

43. Rebuilding plans after ownership/count changes

alvine/spectral/AlvineSpectralSetup3D.hpp:42 — rebuildNUFFTPlans3D(); alvine/spectral/SFSLCoupledStrang3D.hpp:67 — stage update

```
pc->update();

this->rebuildNUFFTPlans3D();

std::make_shared<Nufft_t>(FL, pc->getLocalNum(), …);
```

Reconstructs plans using the current local particle count after migration/reset. This keeps GPU/distributed transform state consistent with ownership changes, although repeated plan creation can be expensive.

44. Reusing configured dependency targets

alvine/CMakeLists.txt:13 — executable target setup

```
find_package(Heffte CONFIG REQUIRED)

target_link_libraries(… ippl MPI::MPI_CXX Heffte::Heffte)
```

Reuses configured IPPL, MPI, and heFFTe targets. alvine does not independently choose CUDA/HIP or force a CPU FFT library here; backend capabilities come from the parent build and dependencies.

45. Inherited runtime lifecycle

alvine/executables/VortexInFourier3D.cpp:19; other executable main() functions

```
ippl::initialize(argc, argv);

…

ippl::finalize();
```

Uses the inherited runtime lifecycle that initializes/finalizes MPI and Kokkos. Device/runtime setup is not reimplemented in alvine.

5. Time integration and retention of device state

46. Parallel attribute-expression integration

alvine/VortexInCellManager.h:557 — LeapFrogStep(); 2D RK4 functions in FslManager.h, SpectralFSLManager.h, VortexInFourierManager.h

```
pc->R_old = pc->R;

pc->R = R_old_temp + 2 * pc->P * this->dt_m;
```

High-level attribute expressions invoke inherited parallel assignment. These statements are not serial host loops despite lacking an explicit local parallel_for. RK4 base/slopes are likewise managed attributes.

47. 3D RK4 slope and stage kernels

alvine/VortexInFourier3DManager.h:1689 — storeRK4OmegaRHS(), setRK4StageState(), finalizeRK4State()

```
rhs(p)[0] = omegaX * duxdx(p) + omegaY * duxdy(p)

          + omegaZ * duxdz(p);

R(p)[d] = baseRView(p)[d] + dtScale * kRView(p)[d];
```

Explicit per-particle kernels form stretching/viscosity slopes, intermediate states, and final RK4 combinations. Base states and all slopes stay in attributes throughout the stages.

48. Coupled position/strength RK4 kernels

alvine/spectral/SFSLCoupledStrang3D.hpp:5 — evaluateCoupledStrangRHS3D(), setCoupledStrangStage3D(), advanceCoupledNonlinearRK4_3D()

```
rhs(p)[0] = stretching

    ? w(p)[0]*xx(p)+w(p)[1]*xy(p)+w(p)[2]*xz(p) : T(0);

R(p)[d] = R0(p)[d] + h*rRHS(p)[d];

w(p)[d] = w0(p)[d] + h*wRHS(p)[d];
```

Couples position and strength evolution in device kernels. Each stage evaluates its own particle state through spectral scatter/gather. Registered base states and slopes migrate with particles.

49. Strang splitting without host scratch state

alvine/spectral/SFSLStrang3D.hpp:6 — applyStrangSpectralDiffusion3D(), accumulateStrangStretchingStage3D(), setStrangAdvectionStage3D(), advanceStrangAdvectionRK4_3D()

```
"sfsl_strang_spectral_diffusion"

"sfsl_strang_rk4_spectral_stage"

R(p)[d] = base(p)[d] + interval * rhs(p)[d];
```

Diffusion and spectral stretching stages use grid kernels; advection stages and final combinations use particle kernels. Spectral base states are preserved with Kokkos::deep_copy, avoiding host scratch state.

50. Euler grid source and IMEX spectral update

alvine/spectral/SFSLEuler3D.hpp:17 — computeEulerGridSource3D(), updateEulerSpectralVorticity3D()

```
"sfsl_euler_grid_stretching"

const T denominator = T(1) + nu * dt * k2;

qx(i,j,k) = (qx(i,j,k) + dt * sx(i,j,k) / k2)

          / denominator;
```

Forms grid stretching and the IMEX spectral update in Kokkos kernels. The host controls step sequencing; the per-cell numerical work remains device-capable.

51. Direct IFFT lattice sampling

alvine/spectral/SFSLEuler3D.hpp:149 — sampleEulerLatticeFromGrid3D()

```
omega(p)[d] = wg(i,j,k)[d] * volume;

P(p)[d] = ug(i,j,k)[d];
```

Directly samples the reconstructed IFFT lattice in a particle kernel. This path explicitly avoids an additional CIC or type-2 NUFFT evaluation.

52. Retaining spectral state to avoid redundant transforms

alvine/spectral/SFSLEuler3D.hpp:186 — advectSpectralEuler3D()

```
// no redundant FFT of the reconstructed input grid is necessary.

this->computeSpectralVelocityModes3D(false);
```

Retains spectral state between steps and avoids transforming reconstructed input back to modes unnecessarily. This reduces transform work, device memory traffic, and distributed FFT communication.

53. Same-space vorticity preservation around the solver

alvine/FslManager.h:568 — advectForward(); VIC initialization/advance/dump paths

```
auto omega_n = omegaField.deepCopy();

… runSolver(); …

Kokkos::deep_copy(omegaField.getView(), omega_n.getView());
```

Saves/restores vorticity around a solver that overwrites the field with stream function. These are same-space field copies, not implicit device-to-host transfers.

54. Same-space diagnostic state restoration

alvine/VortexInFourier3DManager.h:425 — diagnostic state preservation

```
Kokkos::deep_copy(this->omega_x_hat_m.getView(),

                  omegaXState.getView());
```

Diagnostic comparison paths preserve and restore mode fields on their existing memory space instead of reconstructing solver state on the host.

6. Reductions, atomics, MPI boundaries, and synchronization

55. Device conservation and bulk diagnostics

alvine/diagnostics/AlvineDiagnostics.hpp:8 — computeParticleCirculation(), computeGridCirculation(), computeKineticEnergy(), computeEnstrophy()

```
Kokkos::parallel_reduce(…,

    KOKKOS_LAMBDA(…, double& lsum) { lsum += …; }, local);

ippl::Comm->reduce(local, global, 1, std::plus<double>());
```

Reduces full particle/grid data on the execution backend, then communicates a host scalar. The full arrays are not copied to the host for conservation diagnostics.

56. Device debug reductions and host scalar MPI

alvine/FslManager.h:117; alvine/SpectralFSLManager.h:136 — computeOmegaL2(), logPushDebug(), logVelocityRatioDebug()

```
Kokkos::Max<double>(localMaxVel)

Kokkos::Max<double>(localMaxDx)

MPI_Allreduce(&localMaxRatio, &globalMaxRatio, …);
```

Computes sums/maxima from device Views, including multiple results in one traversal. MPI consumes the small host results, so these diagnostics do not require GPU-aware MPI. logPushDebug() in these files assigns its “global” maxima from local maxima; that particular output is rank-local.

57. Parallel particle enstrophy

alvine/VortexInFourierManager.h:417 — computeParticleEnstrophyFromStrengths()

```
"vif_particle_enstrophy_from_strengths"

KOKKOS_LAMBDA(const size_t p, double& lsum) { … }
```

Computes particle-based enstrophy through a parallel reduction without exporting particle strengths.

58. 2D spectral diagnostics in mode space

alvine/diagnostics/AlvineSpectralDiagnostics.hpp:27 — computeSpectralDivergenceL2(), computeSpectralEnergy(), computeSpectralEnstrophy()

```
"compute_spectral_divergence_l2"

"compute_spectral_energy"

"compute_spectral_enstrophy"
```

Mode-space diagnostics use MDRange reductions on spectral Views rather than IFFT-plus-host analysis.

59. 3D spectral diagnostics and multiple scalar results

alvine/diagnostics/AlvineSpectralDiagnostics3D.hpp:25 — energy/enstrophy, velocity/vorticity divergence, TGV spectral projection functions

```
Kokkos::parallel_reduce(…, policy_type(…), KOKKOS_LAMBDA(…),

    Kokkos::Sum<double>(localDot),

    Kokkos::Sum<double>(localRef2));
```

Extends device reduction to 3D diagnostics. Projection comparisons compute multiple scalar statistics in a single mode traversal.

60. Device analytic-error evaluation

alvine/diagnostics/AlvineDiagnostics.hpp:141 — logTgvVelocityDiagnostics();
alvine/diagnostics/AlvineTaylorGreenDiagnostics3D.hpp:4 — TGV error/projection/component diagnostics

```
const auto exact = TaylorGreen3D<T>::velocity(x,y,z);

err2 += …;

ref2 += …;
```

Computes reference values and errors inside device reductions. Exact reference arrays do not need to be generated on the host and uploaded.

61. Device deformation reduction for adaptive timesteps

alvine/diagnostics/AlvineSpectralDiagnostics3D.hpp:547 — computeParticleDeformationOneNorm3D();
alvine/spectral/SFSLEuler3D.hpp:77 — grid deformation calculation

```
Kokkos::Max<double>(localMax)

ippl::Comm->allreduce(localMax, globalMax, 1,
```

```
                      std::greater<double>());
```

Adaptive LCFL calculation reduces deformation on-device and transfers/communicates only a scalar maximum. GPU-computed information drives the host timestep without downloading gradients.

62. Atomic device spectrum histograms

alvine/diagnostics/AlvineSpectralDiagnostics.hpp:171 — computeSpectralVorticitySpectrum();
alvine/diagnostics/AlvineSpectralDiagnostics3D.hpp:163 — 3D counterpart

```
Kokkos::View<double*> localSpectrum(…, shellCount);

Kokkos::deep_copy(localSpectrum, 0.0);

Kokkos::atomic_add(&localSpectrum(shell), …);

Kokkos::atomic_add(&localModeCounts(shell), std::uint64_t(1));
```

Shell bins receive contributions from many concurrent modes, so atomics prevent lost updates. Bins reside in default-space Views and are initialized before accumulation. Floating-point sums are order-dependent; low-shell contention can affect performance.

63. Atomic spectrum metrics in the 3D VIF manager

alvine/VortexInFourier3DManager.h:319 — computeCurrentVorticitySpectrumShellMetrics3D()

```
Kokkos::atomic_add(&shellEnstrophy(shell),

                   0.5 * volume * omegaAmp);

Kokkos::atomic_add(&shellCounts(shell), std::uint64_t(1));
```

Another explicit device spectrum histogram, using the same concurrency-safe accumulation pattern.

64. Compact spectrum transfers before MPI

Spectrum functions above

```
Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(),

                                    localSpectrum);

ippl::Comm->allreduce(localValues.data(), globalValues.data(),

                      shellCount, …);
```

Transfers the compact shell arrays rather than the complete Fourier fields. Subsequent MPI and result formatting operate on host buffers.

65. Device RHS consistency verification

alvine/VortexInFourier3DManager.h:437 — RHS consistency functions

```
"compute_grid_rhs_reference_3d"

"pack_particle_rhs_for_scatter_3d"

"restore_particle_omega_after_rhs_consistency_3d"

"compute_rhs_consistency_metrics_3d"
```

Reference RHS formation, temporary particle conversion, restoration, field addition, and error reductions are device kernels. Verification avoids turning the numerical comparison itself into a host-only path.

66. Explicit completion before dependent operations

Particle initialization, deletion, pushing, spectral filtering/packing, and integration functions throughout alvine

```
Kokkos::fence();

pc->update();
```

Explicitly completes asynchronous kernels before migration, destruction, external transforms, or dependent orchestration. Fences improve ordering clarity but globally synchronizing after many kernels limits overlap.

67. Distributed load-balancing orchestration

alvine/LoadBalancer.hpp:38 — updateLayout(), repartition(), balance()

```
omega_m->updateLayout(*fl);

pc_m->update();

MPI_Allgather(&local, …, res.data(), …);
```

Delegates field redistribution and particle movement to IPPL. The imbalance decision communicates host metadata, not particle arrays. This is distributed orchestration, not a custom GPU load-balancing kernel.

68. Migration-safe RK4 histories

alvine/particles/ParticleContainer3D.hpp:75 — registerAttributes(); coupled RK4 stage functions

```
this->addAttribute(rk4_R0);

this->addAttribute(rk4_omega_k1);
```

```
// Registered base states and all slopes migrate with the particles.
```

Stage histories follow particle ownership and permutation changes. This is crucial for correct MPI/GPU RK4 execution; separately allocated unregistered history arrays would lose correspondence after migration.

7. Explicit host-transfer and CUDA-debug boundaries

69. Explicit field downloads for output

alvine/VtkDump.hpp:54 — writeScalarField2D(), writeScalarFieldCsv2D(), writeVectorField2D(), writeVectorField3D(), writeFourierMagnitudeField2D()

```
auto host = field.getHostMirror();

Kokkos::deep_copy(host, field.getView());
```

Output deliberately stages device data to host-accessible storage before stream/file operations. The host loops read mirrors, not device-only pointers. These operations incur full-field transfers.

70. Explicit particle downloads for output

alvine/VtkDump.hpp:301 — writeParticles2D(); VIC/VIF dumpParticleDataPerRank()

```
auto rHost = particles.R.getHostMirror();

Kokkos::deep_copy(rHost, particles.R.getView());

Kokkos::deep_copy(omegaHost, particles.omega.getView());
```

Particle output explicitly downloads positions and strengths before host formatting.

71. MPI-coordinated filesystem operations

alvine/VtkDump.hpp:46 — ensureOutputDirectory(), filename helpers

```
if (ippl::Comm->rank() == 0) {

    …create_directories(…);

}

ippl::Comm->barrier();
```

Coordinates host filesystem work and rank-specific output. The MPI barrier is process synchronization; it should not be interpreted as a device fence.

72. CUDA asynchronous-error checkpoints

alvine/VortexInFourier3DManager.h:1006 — traceCudaCheckpoint3D()

```
#if defined(KOKKOS_ENABLE_CUDA)
```

```
const cudaError_t err = cudaDeviceSynchronize();

… cudaGetErrorString(err) …

#else

Kokkos::fence();

#endif
```

Explicit CUDA debugging support. Synchronization exposes asynchronous launch/memory failures at named scatter checkpoints, with a portable fence fallback. It is gated by tracing settings and sacrifices overlap when enabled.

73. CUDA memory reporting and debugger hooks

alvine/VortexInFourier3DManager.h — traceCudaMemoryInfo3D() and checkpoint helpers

```
cudaMemGetInfo(&freeBytes, &totalBytes);

extern "C" ALVINE_CUDA_GDB_NOINLINE inline void …
```

Reports GPU memory state and exposes stable named debugger checkpoints around component type-1 NUFFTs. These are GPU observability decisions, not compute acceleration.

74. Explicit downloads in particle/mode traces

alvine/VortexInFourier3DManager.h — dumpRemeshPreScatterParticlePositions3D(), traceParticles3D(), traceRawSpectralVorticityModes3D(), traceModes3D()

```
auto Rhost = pc->R.getHostMirror();

Kokkos::deep_copy(Rhost, pc->R.getView());
```

Trace output uses explicit host mirrors. The particle trace downloads many attributes; the mode trace downloads several spectral fields, so tracing can significantly change timing and memory-transfer volume.

75. Trace-path preparation stays in kernels

alvine/VortexInFourier3DManager.h — tracedSpectralScatter3D(), zeroParticleViscosityForTrace3D()

```
"trace_split_omega_components_3d"

"trace_precondition_spectral_vorticity_modes_3d"

"zero_particle_viscosity_for_trace_3d"
```

Even diagnostic-path state preparation remains in device kernels; only the subsequent inspection/printing crosses to the host.

8. GPU-portable tests and incomplete/legacy implementations

76. Execution-space regression checks

alvine/tests/VIFEuler3DTest.cpp:20 — setShear(), stateError(); alvine/tests/SFSL3DTestSupport.hpp:8 — setShear(), gridError(), coupledStageError(), modeError()

```
KOKKOS_LAMBDA(…) { … }

Kokkos::parallel_reduce(…, Kokkos::Max<T>(…));

MPI_Allreduce(…);
```

Test initialization and error measurement are execution-space kernels followed by scalar MPI. The tests can exercise GPU-resident states without requiring host access to their underlying Views.

77. Portable host reference calculations in tests

alvine/tests/SFSLEuler3DTest.cpp:21 — cellCenteredFftError() and reference construction

```
Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace{},

                                    modes.getView());

Kokkos::deep_copy(modes.getView(), grid);
```

Some reference calculations are intentionally host loops, but use explicit mirrors and upload/download boundaries. This is portable test code, not a device-only test implementation.

78. Legacy uppercase FSL implementation

alvine/FSLManager.h:106 — initializeVirtualParticles(), initializeGridVorticity(), computeOmegaL2(), pushVirtualParticlesForward(), clearVirtualParticles()

```
MDRangePolicy<Rank<2>>({i0,j0},{i1+1,j1+1})

KOKKOS_LAMBDA(…) { … }
```

Contains the same portable loop/reduction/mask patterns, but uses global domain indices directly for local field Views in several places. That is a distributed-indexing caveat. The built FSL executable uses the distinct FslManager.h implementation.

79. Legacy device-lambda host-object access

alvine/FSLManager.h — logPushDebug()

```
// Inside KOKKOS_LAMBDA:
```

```
double dx = ux * this->dt_m;
```

Captures/accesses the host manager through this; a device annotation does not make that host object device-accessible. The current FslManager.h extracts dt before its kernel.

80. Incomplete backward semi-Lagrangian device skeleton

alvine/BackwardSemiLagrangianManager.h:90 — initializeGridVorticity(), initialOmega(), advectVorticityBSL()

```
KOKKOS_INLINE_FUNCTION static T initialOmega(…)

KOKKOS_LAMBDA(…) { … omega_new(i,j) = …; }
```

Shows device-callable initialization and parallel backward tracing/interpolation with separate old/new Views to avoid in-place races. It remains a skeleton: old/new fields and some bounds are undeclared, and global departure indices are not translated into a distributed local sampling scheme. It is not evidence of a working GPU BSL solver.

81. Annotated distribution functors with host storage

alvine/VortexDistributions.h:14 — BaseDistribution, UnitDisk::operator(), HalfPlane::operator(), Band::operator()

```
view_type r;

host_type omega;

KOKKOS_INLINE_FUNCTION void operator()(…) const { … }
```

Device annotations show portability intent, but omega is explicitly a host-mirror type. That storage is not generally accessible from a discrete GPU. Virtual dispatch and std::pow/std::sqrt add further portability concerns; these functors should not be counted as validated device execution.

82. 2D viscosity kernel host-object capture risk

alvine/spectral/AlvineSpectralOps2D.hpp:389 — computeSpectralViscosity()

```
// Inside KOKKOS_LAMBDA:

visc(i,j) = … * viscosity_m * …;
```

Another direct manager-member access inside a device lambda. Other spectral kernels copy parameters to local scalars first; this one is a GPU-portability risk unless the manager is device-accessible.

9. Functionality inherited from IPPL/Kokkos

These mechanisms enable the alvine decisions above, but their implementations live outside alvine.

83. View storage, spaces, and default layouts

src/Types/ViewTypes.h:44 — ippl::detail::ViewType; ParticleAttrib/BareField storage

```
typedef Kokkos::View<typename NPtr<T, Dim>::type,

                     Properties...> view_type;
```

Reference-counted Views, selected memory/execution spaces, and default layout. alvine does not implement GPU allocation or explicitly choose LayoutLeft/LayoutRight for its ordinary particle/grid containers.

84. View-specific range-policy construction

src/Utility/ParallelDispatch.h:55 — getRangePolicy()

```
using exec_space = typename View::execution_space;

begin[d] = shift;

end[d] = view.extent(d) - shift;
```

View-specific execution policy and ghost exclusion.

85. Expression-template assignments execute in kernels

src/Particle/ParticleAttrib.hpp:105 — scalar/expression operator=(); src/Field/BareField.hpp:190 — expression assignment

```
Kokkos::RangePolicy<execution_space>

Kokkos::parallel_for(…, KOKKOS_LAMBDA(…) { … });
```

Device execution of statements such as pc->R = …, omegaField = omegaField / dA, and attribute zeroing. Expression-template capture and evaluation are IPPL functionality.

86. Deep copies and host mirrors

src/Field/BareField.hpp:112 — deepCopy(); src/Field/BareField.h:172 — getHostMirror()

```
Kokkos::deep_copy(copy.dview_m, dview_m);

Kokkos::create_mirror(dview_m)
```

Same-space copies and creation of host-accessible mirrors. Kokkos performs the actual copying and synchronization semantics.

87. Inherited CIC interpolation

alvine/particles/AlvineParticleGridTransfer.hpp:8 calls inherited gather(); alvine/particles/AlvineScatterCIC.hpp:11 calls inherited scatter()

```
gather(pc->P, uField, pc->R);

scatter(pc->omega, omegaField, pc->R);
```

CIC interpolation kernels, particle sorting/configuration, execution policies, and halo handling are IPPL's implementation. alvine chooses when to call them and how to normalize/check results.

88. Inherited deposition atomics

src/Interpolation/CIC.hpp:31 — scatterToPoint(); current scatter facade/implementations

```
Kokkos::atomic_add(&view(…), val * …);
```

Concurrent deposition protection. These interpolation atomics are inherited, unlike the explicit spectrum atomics implemented in alvine.

89. Inherited tiled scatter teams and scratch memory

src/Interpolation/Scatter/TiledScatter.h:43 — tiled scatter implementation

```
using team_policy = Kokkos::TeamPolicy<execution_space>;

using scratch_space = typename execution_space::scratch_memory_space;

set_scratch_size(0, Kokkos::PerTeam(scratch))
```

Team-level parallelism, scratch histograms, scratch atomics, and global flushing used by alvine's tiled NUFFT choice. Tile/team sizing and scratch-capacity handling reside in IPPL.

90. Inherited sorted gather and halo preparation

src/Interpolation/Gather/AtomicGather.h:3; src/Interpolation/Gather/Gather.h:121

```
// … neither … actually issues atomic operations.

field.fillHalo();
```

Sorted/unsorted particle gather and automatic halo preparation. atomic_sort is a method name, not evidence of atomic writes during gathering.

91. Inherited halo packing and MPI archive transport

src/Field/HaloCells.hpp:175 — pack(), unpack(); src/Communicate/Communicator.h:186 — archive communication

```
Kokkos::parallel_for("HaloCells::pack()", …);

Kokkos::fence();

MPI_Isend(ar.getBuffer(), ar.getSize(), MPI_BYTE, …);
```

Execution-space packing/unpacking, completion fences, and MPI transport of archive buffers. alvine's fillHalo() delegates this entire boundary. Device-buffer transport requires suitable MPI support.

92. Inherited particle migration and attribute packing

src/Particle/ParticleSpatialLayout.hpp:78 — ownership/count exchange and migration; src/Particle/ParticleAttrib.hpp:76 — pack(), unpack()

```
MPI_Isend(d_sendCounts + r, …);

MPI_Alltoall(rankSendCount_d_.data(), …);

Kokkos::parallel_for("ParticleAttrib::pack()", …);
```

Device ownership bookkeeping, selectable count-exchange paths, host copies of routing metadata, and parallel attribute serialization/migration. pc->update() is not an alvine MPI/GPU implementation.

93. Inherited mask processing and compaction

src/Particle/ParticleBase.hpp:215 — destroy(); ParticleAttrib::destroy()

```
Kokkos::deep_copy(locDeleteIndex, -1);

Kokkos::parallel_for("ParticleAttrib::destroy()", …);
```

Device mask processing and attribute compaction behind alvine's deletion calls.

94. Inherited native NUFFT engine

src/FFT/Transform/NUFFT.hpp:251 — initNative(), transformNative(); src/FFT/NUFFT/NativeNUFFT.h:230 — type1(), type2()

```
Q.scatter_kernel(*grid_field_, R, kernel_, cfg_.scatter_config);

performPrunedFFT(1, f);

Q.gather(*grid_field_, R, kernel_, false, cfg_.gather_config);
```

Native NUFFT spreading, correction/deconvolution kernels, oversampled working storage, pruned FFTs, gathering, halo communication, and inter-stage fences. alvine selects/configures and invokes this engine.

95. Inherited memory-space FFT backend dispatch

src/FFT/Traits.h:195 — HeffteBackend specializations

```
HeffteBackend<Kokkos::CudaSpace> … heffte::backend::cufft

HeffteBackend<Kokkos::HIPSpace> … heffte::backend::rocfft
```

GPU FFT backend selection from memory space. HIP requires the relevant heFFTe support; the inspected SYCL specialization documents a stock CPU fallback. Portable Kokkos kernels therefore do not guarantee GPU FFT execution on every backend.

96. Inherited FFT options, workspaces, and normalization

src/FFT/Backend/Heffte.h:98 — makeHeffteOptions(), transform wrappers

```
opts.use_gpu_aware = true;

opts.algorithm = heffte::reshape_algorithm::p2p_plined;

heffte_->forward(in, out, workspace_.data(), heffte::scale::full);
```

Converts alvine’s plan settings into heFFTe options, owns FFT workspaces, and supplies transform normalization. Actual FFT and redistribution work is performed by heFFTe/backend libraries.

97. Inherited FFT data-layout adaptation

src/FFT/Transform/CC.h:45 — FFT scratch type; src/FFT/Transform/Common.h:50 — toTemp(), fromTemp()

```
… Kokkos::LayoutLeft, MemSpace …

"FFT_toTemp"

"FFT_fromTemp"
```

Contiguous FFT scratch layout and execution-space copies that strip/restore ghost cells. These are inherited data-layout adaptations, not explicit layout decisions implemented in alvine.

98. Kokkos backend primitives

Kokkos mechanisms used throughout the inspected code

```
KOKKOS_LAMBDA    KOKKOS_INLINE_FUNCTION

parallel_for     parallel_reduce    atomic_add
```

```
deep_copy        fence
```

Device annotations, backend dispatch, reduction implementation, atomics, transfers, and synchronization primitives themselves are Kokkos functionality. alvine supplies the algorithms and chooses where to use them.

Portability and numerical limits

The principal limits are the noted host-object captures, legacy host-mirror distribution storage, and unfinished BSL code. Otherwise, the inspected compute paths consistently avoid host traversal of particle/grid/mode arrays. Floating-point reductions, deposition, and spectrum atomics can change accumulation order across OpenMP/GPU backends and MPI decompositions; source portability does not imply bitwise reproducibility or demonstrated GPU performance.